



4.1 SQL简介



SQL简介

- 维基百科的定义：
 - SQL (Structured Query Language, 结构性查询语言) 是一种特定目的编程语言, 用于管理关系数据库管理系统, 或在关系流数据管理系统中进行流处理。
 - SQL基于关系代数和元组关系演算, 包括一个数据定义语言和数据操作语言。SQL的范围包括数据插入、查询、更新和删除, 数据库模式创建和修改, 以及数据访问控制。



DDL

- DDL (Data Definition Language) 数据定义语言
 - 用于定义或修改数据库中的对象。如：表、索引、视图、数据库、序列、用户、角色、表空间、存储过程等。
 - (1) 定义数据库
 - (2) 定义表空间
 - (3) 定义表
 - (4) 定义分区表
 - (5) 定义索引
 - (6) 定义存储过程
 - (7) 定义函数和视图



DCL

- DCL (Data Control Language) 数据控制语言
 - 用来设置或更改数据库事务、授权操作（用户或角色授权，权限回收，创建角色，删除角色等）、锁表（支持SHARE和EXCLUSIVE两种锁表模式）、停机等。



DML

- DML (Data Manipulation Language) 数据操纵语言
 - 用于对数据库表中的数据进行操作，如插入，更新和删除。



DQL

- DQL (Data Query Language) 数据查询语言
 - 用来查询数据库内的数据，如查询数据、合并多个select语句的结果集。



目录

1. openGauss SQL基础
- 2. 数据查询**
3. SQL编程



简单查询

- 语法格式:

```
SELECT [/*+ plan_hint */] [ ALL | DISTINCT [ ON ( expression [, ...] ) ] ]  
{ * | {expression [ [ AS ] output_name ]} [, ...] }  
[ FROM FROM_item [, ...] ]  
[ WHERE condition ]  
[ GROUP BY grouping_element [, ...] ]  
[ HAVING condition [, ...] ]  
[ ORDER BY {expression [ [ ASC | DESC | USING operator ] | nlssort_expression_clause } [ NULLS { FIRST  
| LAST } ]} [, ...] ]  
[ LIMIT { [offset,] count | ALL } ]  
[ OFFSET start [ ROW | ROWS ] ]  
[ , ... ]
```

- 使用方法:

- SELECT关键字之后和FROM子句之前出现的表达式称为SELECT项。SELECT项用于指定要查询的列，FROM指定要从哪个表中查询。



示例

- 示例：SELECT后面使用*号查询bank_card表中的所有列。

```
SELECT * FROM bank_card;
```

b_number	b_type	b_c_id
6222021302020000001	Credit Card	1
6222021302020000002	Credit Card	3
6222021302020000003	Credit Card	5



去除重复值（1）

- DISTINCT关键字

- 从SELECT的结果集中删除所有重复的行，使结果集中的每行都是唯一的。取值范围：已存在的字段名，或字段表达式。

- 语法格式

- 如果在DISTINCT关键字后只有一个列，则使用该列来计算重复，如果有两列或者多列，则将使用这些列的组合来进行重复检查。



去除重复值（2）

- 利用distinct关键字来查询员工的岗位和奖金，去除岗位和奖金相同的记录。

staff_id	name	job	bonus
30	wangxin	developer	9000
31	xufeng	tester	7000
34	denggui	tester	7000
35	caoming	developer	10000
37	lixue	developer	9000

```
select distinct job, bonus from sections;
```

```
JOB          BONUS
```

```
-----
```

```
developer    9000
```

```
tester       7000
```

```
developer    10000
```

```
3 rows fetched.
```



查询列的选择

- 在选择查询列时，列名可以用下面几种形式表达：

- 手动输入列名，多个列之间用英文逗号 (,) 分隔。

```
SELECT a, b, f1, f2 FROM t1, t2;
```

其中，列a、b是表t1中的列，f1、f2是表t2中的列。

- ```
SELECT a+b FROM t1;
```

- 如果某两个或某几个表正好有一些共同的列名，推荐使用表名限定列名。不限定列名可以得到查询结果，但使用完全限定的表和列名称，可以减少数据库内部的处理工作量，从而提升查询的返回性能。例如：

```
SELECT t1.f1, t2.f1 FROM t1, t2;
```



## 示例

- 示例：查看bank\_card表中卡号和卡类型信息。

```
postgres=# SELECT b_number,b_type FROM bank_card;
```

| b_number            | b_type      |
|---------------------|-------------|
| 6222021302020000001 | Credit Card |
| 6222021302020000002 | Credit Card |
| 6222021302020000003 | Credit Card |

- 示例：查看客户为1的客户编号，客户名称，客户邮箱和银行卡号信息。

```
postgres=# SELECT a.c_id,a.c_name, a.c_mail, b.b_number FROM client a, bank_card b where
a.c_id= 1 and b.b_c_id = 1;
```

| c_id | c_name   | c_mail             | b_number            |
|------|----------|--------------------|---------------------|
| 1    | Zhang Yi | zhangyi@Huawei.com | 6222021302020000001 |



# 别名

- 通过使用子句AS some\_name, 可以为表名称或列名称指定另一个标题名显示, 一般创建别名是为了让列名称的可读性更强。
- 语法格式
  - 列和表的SQL别名分别跟在相应的列名和表名后面, 中间可以加或不加一个“AS”关键字。
- 示例: 别名使用。

```
postgres=# SELECT b_c_id AS CardID, b_type CardType FROM bank_card;
```

| cardid | cardtype     |
|--------|--------------|
| 1      | Credit Card  |
| 3      | Credit Card  |
| 5      | Savings Card |

```
postgres=# SELECT a.c_id CID ,a.c_name Name, a.c_mail Email, b.b_number "Card Number" FROM client a, bank_card b where a.c_id= 1 and b.b_c_id = 1;
```

| cid | name     | email              | Card Number         |
|-----|----------|--------------------|---------------------|
| 1   | Zhang Yi | zhangyi@Huawei.com | 6222021302020000001 |



# 条件查询 (1)

- 在SELECT语句中，可以通过设置条件以达到更精确的查询。条件由表达式与操作符共同指定，且条件返回的值是TRUE，FALSE或UNKNOWN。查询条件可以应用于WHERE子句，HAVING子句。
- 语法格式：

- condition子句

```
select_statement { predicate } [{ AND | OR } condition] [, ... n]
```

- predicate子句

```
{ expression { = | <> | != | > | >= | < | <= } { ALL | ANY } expression | (SELECT)
| string_expression [NOT] LIKE string_expression
| expression [NOT] BETWEEN expression AND expression
| expression IS [NOT] NULL
| expression [NOT] IN (SELECT | expression [, ... n])
| [NOT] EXISTS (SELECT)
}
```



## 条件查询 (2)

- 查询条件由表达式和操作符共同定义。常用的条件定义方式如下：
  - 比较操作符 “> , < , >= , <= , != , <> , =” 指定的比较查询条件。当查询条件中和数字比较，可以使用单引号引起，也可以不用，当和字符及日期类型的数据比较，则必须用单引号引起。
  - 测试运算符指定的范围查询条件。如果希望返回的结果必须满足多个条件，可以使用AND逻辑操作符连接这些条件；如果希望返回的结果满足多个条件之一即可，可以使用OR逻辑操作符连接这些条件。
- 示例：使用比较操作符来指定查询条件，例如查询信用卡类型的银行卡信息。

```
postgres=# SELECT * FROM bank_card WHERE b_type='Credit Card';
```

| b_number            | b_type      | b_c_id |
|---------------------|-------------|--------|
| 6222021302020000001 | Credit Card | 1      |
| 6222021302020000002 | Credit Card | 3      |
| 6222021302020000003 | Credit Card | 5      |
| 6222021302020000004 | Credit Card | 7      |
| 6222021302020000005 | Credit Card | 9      |





## 条件查询 (3)

- 逻辑操作符

- 常用的逻辑操作符有AND、OR和NOT，他们的运算结果有三个值，分别为TRUE、FALSE和NULL，其中NULL代表未知。他们运算优先级顺序为：NOT > AND > OR。

| a     | b     | a AND b | a OR b | NOT a |
|-------|-------|---------|--------|-------|
| TRUE  | TRUE  | TRUE    | TRUE   | FALSE |
| TRUE  | FALSE | FALSE   | TRUE   | FALSE |
| TRUE  | NULL  | NULL    | TRUE   | FALSE |
| FALSE | FALSE | FALSE   | FALSE  | TRUE  |
| FALSE | NULL  | FALSE   | NULL   | TRUE  |
| NULL  | NULL  | NULL    | NULL   | NULL  |

- 示例：从bank\_card中查询客户ID为1且银行卡类型为信用卡的银行卡信息。

```
postgres=# SELECT* FROM bank_card where b_c_id= 1and b_type='Credit Card';
b_number | b_type | b_c_id
-----+-----+-----
622202130202000001 | Credit Card | 1
```



# join连接查询 (1)

- 实际应用中所需要的数据，经常会需要查询两个或两个以上的表。这种查询两个或两个以上数据表或视图的查询叫做连接查询。连接查询通常建立在存在相互关系的父子表之间。
- 语法格式

```
SELECT [, ...] FROM table_reference
 [LEFT [OUTER] | RIGHT [OUTER] | FULL [OUTER] | INNER]
 JOIN table_reference
 [ON { predicate } [{ AND | OR } condition] [, ... n]]
```

- table\_reference 子句

```
{ table_name [[AS] alias]
 | view_name [[AS] alias]
 | (SELECTquery) [[AS] alias]
}
```



## join连接查询 (2)

- 当查询的FROM子句中出现多个表时，数据库就会执行连接。

- 查询的SELECT列表可以是这些表中任意一些列。

```
SELECT table1.column, table2.column FROM table1, table2;
```

- 大多数连接查询包含至少一个连接条件，连接条件可以在FROM子句中也可以在WHERE子句中。

```
SELECT table1.column, table2.column FROM table1 JOIN table2 ON(table1.column1 =
table2.column2);
SELECT table1.column, table2.column FROM table1, table2 WHERE table1.column1 =
table2.column2;
```

- WHERE子句中可以通过指定 “(+)” 操作符的方法将表的连接关系转换为外连接，但是不建议用户使用这种用法，因为这并不是SQL的标准语法。



## 内连接

- 内连接：内连接的关键字为inner join，其中inner可以省略。使用内连接，连接执行顺序必然遵循语句中所写的表的顺序。
- 示例：查询客户ID、银行卡卡号和银行卡类型。使用client和bank\_card两个相关的列c\_id做查询操作。

```
postgres=# SELECT c.c_id, b.b_number, b.b_type FROM client c JOIN bank_card b ON (b.b_c_id = c.c_id);
```

| c_id | b_number            | b_type      |
|------|---------------------|-------------|
| 1    | 6222021302020000001 | Credit Card |
| 3    | 6222021302020000002 | Credit Card |
| 5    | 6222021302020000003 | Credit Card |
| 7    | 6222021302020000004 | Credit Card |
| 9    | 6222021302020000005 | Credit Card |
| 10   | 6222021302020000006 | Credit Card |
| 12   | 6222021302020000007 | Credit Card |
| 14   | 6222021302020000008 | Credit Card |



# 外连接

- 外连接
  - 内连接所指定的两个数据源处于平等的地位。而外连接不同，外连接以一个数据源为基础，将另一个数据源与之进行条件匹配。
  - 内连接返回两个表中所有满足连接条件的数据记录。外连接不仅返回满足连接条件的记录，还将返回不满足连接条件的记录。
  - 外连接又分为左外连接、右外连接和全外连接。

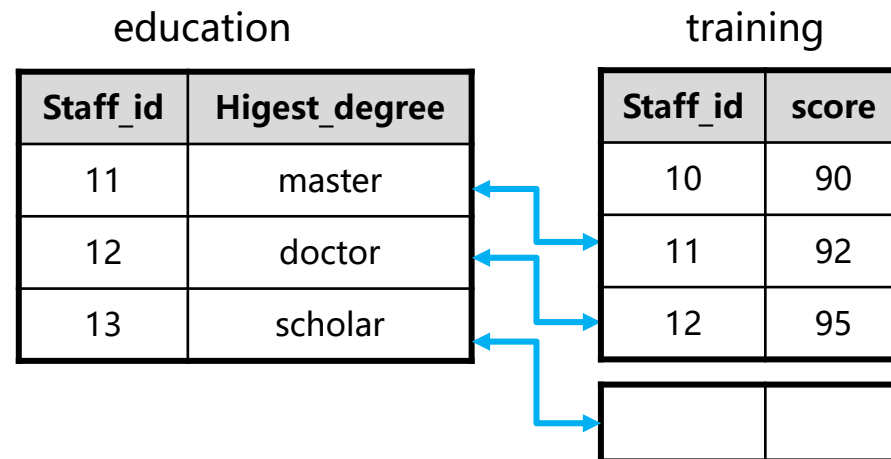


# 左外连接

- 左外连接：
  - 又称左连接，是指以左边的表为基础表进行查询。
  - 根据指定连接条件关联右表，获取基础表以及和条件匹配的右表数据，未匹配条件的右表对应的字段位置填上NULL。
- 示例：

```
postgres=# SELECT e.staff_id, e.highest_degree, t.score
FROM education e LEFT JOIN training t ON (e.staff_id =
t.staff_id);
```

| STAFF_ID | HIGHEST_DEGREE | SCORE |
|----------|----------------|-------|
| 11       | master         | 92    |
| 12       | doctor         | 95    |
| 13       | scholar        |       |



| Staff_id | Highest_degree | score |
|----------|----------------|-------|
| 11       | master         | 92    |
| 12       | doctor         | 95    |
| 13       | scholar        |       |

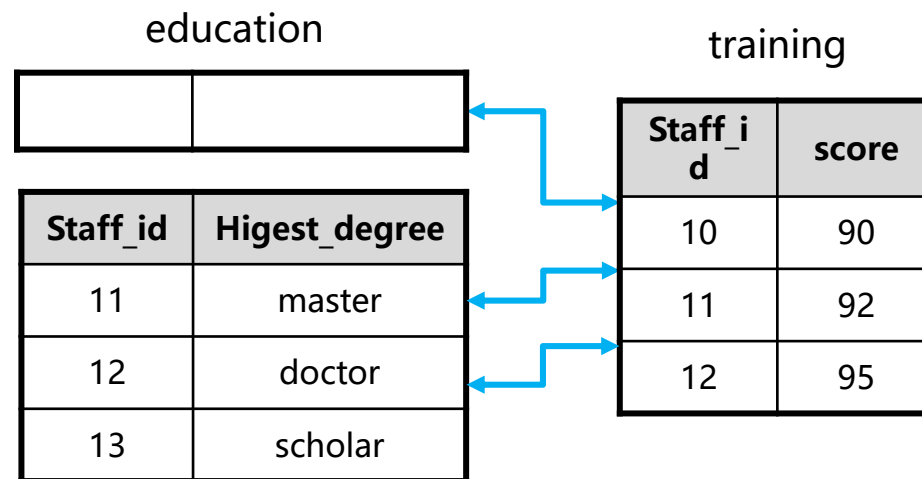


## 右外连接

- 右外连接
  - 又称右连接，是指以右边的表为基础表，在内连接的基础上也查询右边表中有记录，而左边的表中没有记录的数据（左边用NULL值填充）。
- 示例：

```
postgres=# SELECT e.staff_id, e.highest_degree, t.score
FROM education e RIGHT JOIN training t ON (e.staff_id =
t.staff_id);
```

| STAFF_ID | HIGHEST_DEGREE | SCORE |
|----------|----------------|-------|
|          |                | 90    |
| 11       | master         | 92    |
| 12       | doctor         | 95    |



| Staff_id | Highest_degree | score |
|----------|----------------|-------|
|          |                | 90    |
| 11       | master         | 92    |
| 12       | doctor         | 95    |



# 全外连接

- 全外连接
  - 又称全连接，是指除了返回两个表中满足连接条件的记录，还会返回两个表中不满足连接条件的所有其它行（不匹配的另外一边用NULL值填充）。

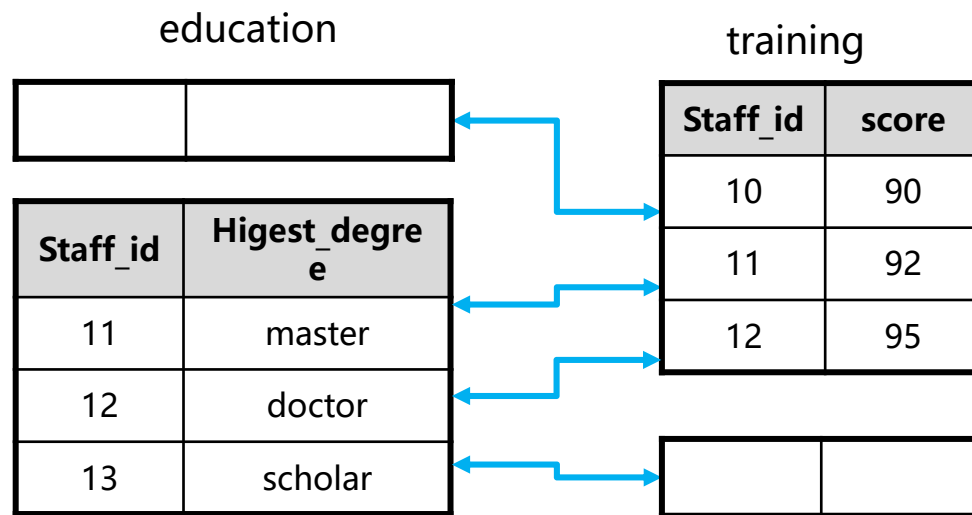
- 示例：

```
postgres=# SELECT e.staff_id, e.highest_degree, t.score
FROM education e FULL JOIN training t ON (e.staff_id =
t.staff_id);
```

| STAFF_ID | HIGHEST_DEGREE | SCORE |
|----------|----------------|-------|
|----------|----------------|-------|

|    |         |    |
|----|---------|----|
| 11 | master  | 92 |
| 12 | doctor  | 95 |
| 13 | scholar |    |

90



| Staff_id | Highest_degree | score |
|----------|----------------|-------|
| 11       | master         | 92    |
| 12       | doctor         | 95    |
| 13       | Scholar        |       |
|          |                | 90    |





## 子查询 (1)

- 子查询或称为内部查询，嵌套查询，指的是在数据库查询的WHERE子句中嵌入查询语句，相当于临时表。一个SELECT语句的查询结果能够作为另一个语句的输入值。
- 以下是子查询必须遵守的几个规则：
  - 子查询必须用括号括起来。
  - 子查询在SELECT子句中只能有一个列，除非在主查询中有多列，与子查询的所选列进行比较。
  - 子查询返回多于一行，只能与多值运算符一起使用，如IN运算符。
  - BETWEEN运算符不能与子查询一起使用，但是，BETWEEN可在子查询内部使用。



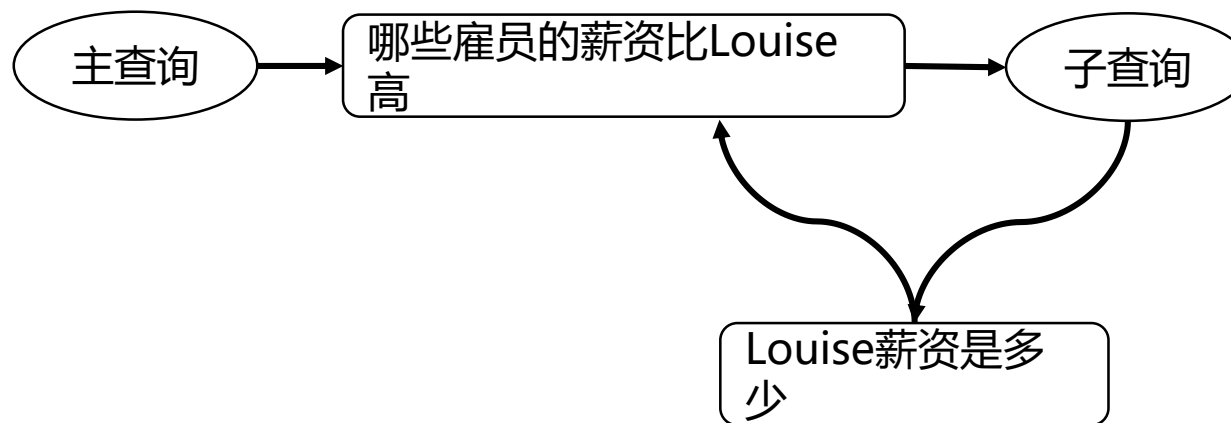
## 子查询 (2)

- 子查询基本语法:

```
SELECT select_list
FROM table
WHERE expr operator
(SELECT select_list FROM table);
```

- 说明:

- 在主查询执行之前先执行子查询;
- 主查询语句中调用子查询的结果。





## 子查询 (3)

- 子查询使用建议：
  - 子查询通常包括在括号里面；
  - 子查询通常放在比较操作符的右边；
  - 子查询中不建议包含ORDER BY子句；
  - 对于单行的子查询使用单行的比较操作符；
  - 对于多行的子查询要使用多行的比较操作符。

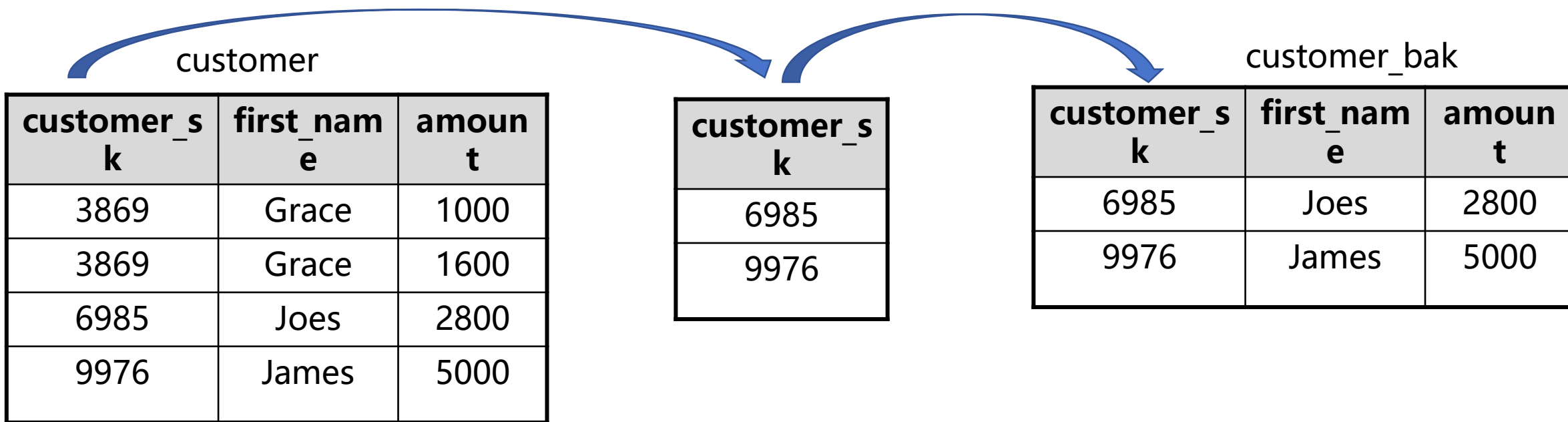
```
openGauss=# BEGIN;
BEGIN
openGauss=# TRUNCATE TABLE staff;
TRUNCATE TABLE
openGauss=# ROLLBACK;
ROLLBACK
openGauss=# SELECT * FROM staff;
id | name | age
----+-----+-----
2 | JUCE | 16
```



# SELECT语句中的子查询使用

- SELECT语句在子查询返回的数据中进行查询。

```
SELECT * FROM customer WHERE customer_sk IN (SELECT customer_sk FROM customer_bak WHERE amount > 2500);
```





## 子查询示例

| 操作符 | 含义          |
|-----|-------------|
| IN  | 等于列表中的某一个值  |
| ANY | 与列表中的任意相值比较 |
| ALL | 与李彪中的所有值相比较 |

- 查询employee\_id在11,101,192,201范围里的雇员信息

```
test=> select employee_id,first_name,hire_date,salary from employees where employee_id in(11,101,192,201);
```

| employee_id | first_name | hire_date           | salary   |
|-------------|------------|---------------------|----------|
| 101         | Neena      | 2005-09-21 00:00:00 | 17000.00 |
| 192         | Sarah      | 2004-02-04 00:00:00 | 4000.00  |
| 201         | Michael    | 2004-02-17 00:00:00 | 13000.00 |

- 查询薪资大于部门为80的任意一个雇员薪资的雇员信息

```
test=> select employee_id,first_name,hire_date,salary from employees
where salary > any (select salary from employees where department_id=80);
```

| employee_id | first_name | hire_date           | salary   |
|-------------|------------|---------------------|----------|
| 100         | Steven     | 2003-06-17 00:00:00 | 24000.00 |
| 101         | Neena      | 2005-09-21 00:00:00 | 17000.00 |
| 102         | Lex        | 2001-01-13 00:00:00 | 17000.00 |
| 103         | Alexander  | 2006-01-03 00:00:00 | 9000.00  |
| 108         | Nancy      | 2002-08-17 00:00:00 | 12008.00 |
| 109         | Daniel     | 2002-08-16 00:00:00 | 9000.00  |
| 110         | John       | 2005-09-28 00:00:00 | 8200.00  |

- 查询薪资大于部门为80的所有雇员薪资的雇员信息

```
test=> select employee_id,first_name,hire_date,salary from employees
where salary > all (select salary from employees where department_id=80);
```

| employee_id | first_name | hire_date           | salary   |
|-------------|------------|---------------------|----------|
| 100         | Steven     | 2003-06-17 00:00:00 | 24000.00 |
| 101         | Neena      | 2005-09-21 00:00:00 | 17000.00 |
| 102         | Lex        | 2001-01-13 00:00:00 | 17000.00 |



# INSERT语句中的子查询使用

- INSERT语句使用子查询返回的数据插入到另一个表中。基本语法如下

- 基本语法如下。

```
INSERT INTO table_name [(column1 [, column2])]
SELECT [*|column1 [, column2]]
FROM table1 [, table2]
[WHERE VALUE OPERATOR]
```

- 创建表customer\_bak，表结构与customer\_t1一致。

```
openGauss=# CREATE TABLE customer_bak (
customer_sk integer, Id char(5),
first_name char(6), Amount integer
);
```

- 将表customer中的数据插入customer\_bak.

```
openGauss=# INSERT INTO customer_bak SELECT * FROM customer WHERE customer_sk IN (SELECT
customer_sk FROM customer);
```



# UPDATE语句中的子查询使用（1）

- INSERT语句使用子查询返回的数据插入到另一个表中。基本语法如下
  - 基本语法如下：

```
UPDATE table SET column_name = new_value
[WHERE OPERATOR [VALUE]
(SELECT COLUMN_NAME
FROM TABLE_NAME
[WHERE])
```

- 把表customer中所有c\_customer\_sk大于4000的客户的amount更新为原来的0.50倍。

```
UPDATE customer SET amount = amount * 0.5 WHERE customer_sk IN (SELECT customer_sk FROM
customer_bak WHERE c_customer_sk > 5000);
```



## UPDATE语句中的子查询使用（2）

- 更新影响2行，更新后表customer数据如下。

| customer_sk | id    | first_name | amount                                     |
|-------------|-------|------------|--------------------------------------------|
| 3869        | hello | Grace      | 1000                                       |
| 3869        |       | Grace      |                                            |
| 6985        | maps  | Joes       | 1100 (因为值customer_sk>5000,对应的amount数值已经减半) |
| 9976        | world | James      | 2500 (因为值customer_sk>5000,对应的amount数值已经减半) |





# 模糊查询

- like模糊匹配查询：用LIKE进行某个字符串值的通配符匹配,选出特定行

```
test=> select employee_id,first_name,last_name,hire_date,salary,manager_id,department_id from employees where first_name like 'S%';
```

| employee_id | first_name | last_name | hire_date           | salary   | manager_id | department_id |
|-------------|------------|-----------|---------------------|----------|------------|---------------|
| 100         | Steven     | King      | 2003-06-17 00:00:00 | 24000.00 |            | 90            |
| 116         | Shelli     | Baida     | 2005-12-24 00:00:00 | 2900.00  | 114        | 30            |
| 117         | Sigal      | Tobias    | 2005-07-24 00:00:00 | 2800.00  | 114        | 30            |
| 123         | Shanta     | Vollman   | 2005-10-10 00:00:00 | 6500.00  | 100        | 50            |
| 128         | Steven     | Markle    | 2008-03-08 00:00:00 | 2200.00  | 120        | 50            |
| 138         | Stephen    | Stiles    | 2005-10-26 00:00:00 | 3200.00  | 123        | 50            |
| 161         | Sarath     | Sewall    | 2006-11-03 00:00:00 | 7000.00  | 146        | 80            |
| 166         | Sundar     | Ade       | 2008-03-24 00:00:00 | 6400.00  | 147        | 80            |
| 173         | Sundita    | Kumar     | 2008-04-21 00:00:00 | 6100.00  | 148        | 80            |
| 192         | Sarah      | Bell      | 2004-02-04 00:00:00 | 4000.00  | 123        | 50            |
| 194         | Samuel     | McCain    | 2006-07-01 00:00:00 | 3200.00  | 123        | 50            |
| 203         | Susan      | Mavris    | 2002-06-07 00:00:00 | 6500.00  | 101        | 40            |
| 205         | Shelley    | Higgins   | 2002-06-07 00:00:00 | 12008.00 | 101        | 110           |

(13 rows)

- 空值限定 is null/is not null

```
test=> select employee_id,first_name,last_name,hire_date,salary,manager_id,department_id from employees where manager_id is null;
```

| employee_id | first_name | last_name | hire_date           | salary   | manager_id | department_id |
|-------------|------------|-----------|---------------------|----------|------------|---------------|
| 100         | Steven     | King      | 2003-06-17 00:00:00 | 24000.00 |            | 90            |

(1 row)



## where子句示例三（比较限定操作符）

- like模糊匹配查询：用LIKE进行某个字符串值的通配符匹配,选出特定行

```
test=> select employee_id,first_name,last_name,hire_date,salary,manager_id,department_id from employees where first_name like 'S%';
```

| employee_id | first_name | last_name | hire_date           | salary   | manager_id | department_id |
|-------------|------------|-----------|---------------------|----------|------------|---------------|
| 100         | Steven     | King      | 2003-06-17 00:00:00 | 24000.00 |            | 90            |
| 116         | Shelli     | Baida     | 2005-12-24 00:00:00 | 2900.00  | 114        | 30            |
| 117         | Sigal      | Tobias    | 2005-07-24 00:00:00 | 2800.00  | 114        | 30            |
| 123         | Shanta     | Vollman   | 2005-10-10 00:00:00 | 6500.00  | 100        | 50            |
| 128         | Steven     | Markle    | 2008-03-08 00:00:00 | 2200.00  | 120        | 50            |
| 138         | Stephen    | Stiles    | 2005-10-26 00:00:00 | 3200.00  | 123        | 50            |
| 161         | Sarath     | Sewall    | 2006-11-03 00:00:00 | 7000.00  | 146        | 80            |
| 166         | Sundar     | Ande      | 2008-03-24 00:00:00 | 6400.00  | 147        | 80            |
| 173         | Sundita    | Kumar     | 2008-04-21 00:00:00 | 6100.00  | 148        | 80            |
| 192         | Sarah      | Bell      | 2004-02-04 00:00:00 | 4000.00  | 123        | 50            |
| 194         | Samuel     | McCain    | 2006-07-01 00:00:00 | 3200.00  | 123        | 50            |
| 203         | Susan      | Mavris    | 2002-06-07 00:00:00 | 6500.00  | 101        | 40            |
| 205         | Shelley    | Higgins   | 2002-06-07 00:00:00 | 12008.00 | 101        | 110           |

(13 rows)

- 空值限定 is null/is not null

```
test=> select employee_id,first_name,last_name,hire_date,salary,manager_id,department_id from employees where manager_id is null;
```

| employee_id | first_name | last_name | hire_date           | salary   | manager_id | department_id |
|-------------|------------|-----------|---------------------|----------|------------|---------------|
| 100         | Steven     | King      | 2003-06-17 00:00:00 | 24000.00 |            | 90            |

(1 row)



# 聚集查询 (1)

- 计算总和-sum(expression)
  - 对于BIGINT输入，输出类型为NUMBER 。
  - 对于SMALLINT或INT输入，输出类型为BIGINT。
  - 对于浮点数输入，输出类型为DOUBLE PRECISION。

```
test=> select sum(salary) from employees;
 sum

691416.00
(1 row)
```



## 聚集查询 (2)

- 计算平均值-avg(expression)
  - 对于任何整数类型输入，结果都是NUMBER类型。
  - 对于任何浮点输入，都是DOUBLE PRECISION类型。

```
test=> select avg(salary) from employees;
 avg

6461.8317757009345794
(1 row)
```



# 聚集查询示例

- 计算最大值和最小值。
  - 参数类型：任意数组、数值、字符串、日期/时间类型
  - 返回类型：与参数数据类型相同

```
test=> select max(salary),min(salary) from employees;
 max | min
-----+-----
 24000.00 | 2100.00
```

- 统计行数
  - count(\*) --> 返回表中的记录行数,包含空行
  - count(1) --> 返回表中的记录行数,包含空行
  - count(expression) --> 返回表中满足expression不为NULL的行

```
test=> select count(*),count(1),count(commission_pct) from employees;
 count | count | count
-----+-----+-----
 107 | 107 | 35
```



# 分组查询 (1)

- 数据库查询中，分组是一个非常重要的应用。分组是指将数据表中的记录以某个或者某些列为标准，值相等的划分为一组。
- 语法格式

```
GROUP BY { column_name } [, ...]
```

- 使用方法
  - GROUP BY子句中的表达式可以包含FROM子句中表，视图的任何列，无论这些列是否出现在SELECT列表中。
  - GROUP BY子句对行进行分组，但不保证结果集的顺序。 要对分组进行排序，请使用ORDER BY子句。
  - GROUP BY后的表达式可以使用括号，如： group by (expr1, expr2)，或者 group by(expr1), (expr2)。 但不支持 group by (expr1, expr2), expr3 格式。



## 分组查询 (2)

- 利用distinct关键字来查询员工的岗位和奖金，去除岗位和奖金相同的记录。

| staff_id | name     | job       | bonus |
|----------|----------|-----------|-------|
| 30       | wangxin  | developer | 9000  |
| 31       | xufeng   | tester    | 7000  |
| 34       | denggui  | tester    | 7000  |
| 35       | caoming  | developer | 10000 |
| 37       | lixue    | developer | 9000  |
| 39       | chenjing | developer | 9000  |

```
select job, bonus, count(staff_id) sum from bonuses_depa
group by(job,bonus) order by sum;
```

| JOB | BONUS | SUM |
|-----|-------|-----|
|-----|-------|-----|

|           |       |   |
|-----------|-------|---|
| developer | 10000 | 1 |
| tester    | 7000  | 2 |
| developer | 9000  | 3 |

3 rows fetched



## 分组查询 (3)

- HAVING子句

与GROUP BY子句配合用来选择特殊的组。HAVING子句将组的一些属性与一个常数值比较，只有满足HAVING子句中条件的组才会被提取出来。

- 语法格式

```
HAVING condition [, ...]
```

- 示例：查询表sections中岗位人数大于3的各岗位员工总数。

```
select job, count(staff_id) from bonuses_depa group by job having count(staff_id) > 3;
```

| JOB       | COUNT(STAFF_ID) |
|-----------|-----------------|
| -----     |                 |
| developer | 4               |





# 数据限制

- 数据限制功能包括两个独立的子句，LIMIT子句和OFFSET子句。

- LIMIT子句允许限制查询返回的行。

```
LIMIT { count | ALL }
```

- OFFSET子句设置开始返回的位置。

```
OFFSET start
```

- 使用方法

- start: 指定在返回行之前要跳过的行数。
  - count: 指定要返回的最大行数。
  - start和count都被指定时，在开始计算要返回的count行之前会跳过start行。
  - LIMIT 5,20与LIMIT 20 OFFSET 5及OFFSET 5 LIMIT 20等效。



## 示例

- 查询下表 insurance 中的保险信息。通过增加 LIMIT 2 OFFSET 1 限定查询时跳过前1行后，查询总共2行数据。

```
postgres=# SELECT i_name, i_id, i_amount, i_person FROM insurance LIMIT 2 OFFSET 1;
```

| i_name             | i_id | i_amount | i_person |
|--------------------|------|----------|----------|
| Life Insurance     | 2    | 3000     | Seniors  |
| Accident Insurance | 3    | 5000     | All      |



# 结果集

- 不同类型的结果集有各自的应用场景，应用程序需要根据实际情况选择相应的结果集类型。在执行SQL语句过程中，都需要先创建相应的语句对象，而部分创建语句对象的方法提供了设置结果集类型的功能。
- 创建一个Statement对象，该对象将生成具有给定类型和并发性的ResultSet对象。

```
createStatement(int resultSetType, int resultSetConcurrency);
```

- 创建一个PreparedStatement对象，该对象将生成具有给定类型和并发性的ResultSet对象。

```
prepareStatement(String sql, int resultSetType, int resultSetConcurrency);
```

- 创建一个CallableStatement对象，该对象将生成具有给定类型和并发性的ResultSet对象。

```
prepareCall(String sql, int resultSetType, int resultSetConcurrency);
```

- 在使用数据库连接完成相应的数据操作后，需要关闭数据库连接。

```
Connection conn = DriverManager.getConnection("url","user","password");
conn.close();
```



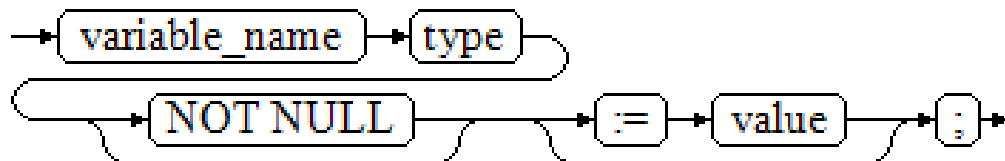
# 目录

1. openGauss SQL基础
2. 数据查询
3. **SQL编程**



# 变量

- 变量声明



- variable\_name, 为变量名。
  - type, 为变量类型。
  - value, 是该变量的初始值（如果不给定初始值, 则初始为NULL）, value也可以是表达式。
- 变量类型除了支持基本类型, 还支持%TYPE和%ROWTYPE类型。
    - my\_name employee.firstname%TYPE
    - my\_employee employee%ROWTYPE



## 基本语句 - 变量作用域

- 变量的作用域：表示变量在代码块中的可访问性和可用性。只有在它的作用域内，变量才有效。
  - 变量必须在declare部分声明，即必须建立BEGIN-END块。块结构也强制变量必须先声明后使用，即变量在过程内有不同作用域、不同的生存期。
  - 同一变量可以在不同的作用域内定义多次，内层的定义会覆盖外层的定义。
  - 在外部块定义的变量，可以在嵌套块中使用。但外部块不能访问嵌套块中的变量。



# BEGIN-END语句

- 在程序中，使用最普遍的逻辑结构就是顺序结构。顺序结构语句的执行过程是从前向后逐条语句依次处理。BEGIN-END语句将多个语句定义成一个语句块，把每个语句块视为一个单元处理。BEGIN-END语句的语法格式如下：

```
BEGIN
```

```
 语句或语句块
```

```
END
```

- BEGIN-END语句的作用类似于 “{ }” 符号，经常在分支结构语句和循环语句中出现，表示语句块的开始和结束。在一个语句块中可以包含另一个语句块。



# 条件语句

- 条件语句的主要作用是判断参数或者语句是否满足已给定的条件，根据判定结果执行相应的操作。
- openGauss有五种形式的IF：

| IF的形式               | 基本概念                                                                 | 示例                                                                                                                                                                                  |
|---------------------|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| IF_THEN::=          | IF_THEN语句是IF的最简单形式。如果条件为真，statements将被执行。否则，将忽略它们的结果使该IF_THEN语句执行结束。 | IF v_user_id <> 0 THEN<br>UPDATE users SET email = v_email WHERE user_id = v_user_id;<br>END IF;                                                                                    |
| IF_THEN_ELSE        | IF_THEN_ELSE语句增加了ELSE的分支，可以声明在条件为假的时候执行的语句。                          | IF parentid IS NULL OR parentid = ''<br>THEN<br>RETURN;<br>ELSE<br>hp_true_filename(parentid);--表示调用存储过程<br>END IF;                                                                 |
| IF_THEN_ELSE IF     | IF语句可以嵌套。                                                            | IF sex = 'm' THEN pretty_sex := 'man';<br>ELSE<br>IF sex = 'f' THEN<br>pretty_sex := 'woman';<br>END IF;<br>END IF;                                                                 |
| IF_THEN_ELSIF_ELSE  | IF_THEN_ELSE语句增加了ELSE的分支，可以声明在条件为假的时候执行的语句。                          | IF number_tmp = 0 THEN result := 'zero';<br>ELSIF number_tmp > 0 THEN result := 'positive';<br>ELSIF number_tmp < 0 THEN result := 'negative';<br>ELSE result := 'NULL';<br>END IF; |
| IF_THEN_ELSEIF_ELSE | ELSEIF是ELSIF的别名。                                                     |                                                                                                                                                                                     |





# 循环语句

- 简单LOOP语句：该循环需要结合EXIT使用，否则将陷入死循环。

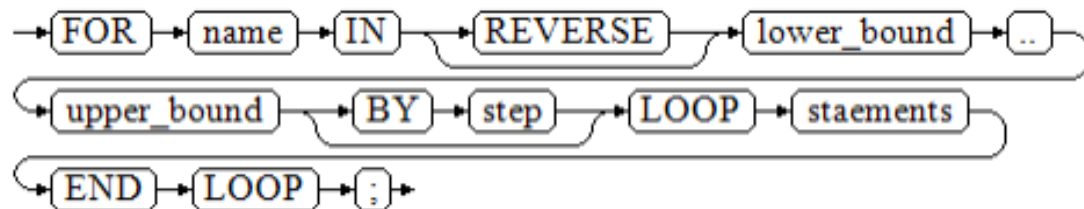


```
CREATE OR REPLACE PROCEDURE proc_loop(i in integer, count out integer) AS
BEGIN
 count:=0;
 LOOP
 IF count > i THEN
 raise info 'count is %. ', count;
 EXIT;
 ELSE
 count:=count+1;
 END IF;
 END LOOP;
END;
/
CALL proc_loop(10,5);
```



# 控制语句 - FOR\_LOOP查询语句 (1)

- FOR\_LOOP (integer变量) 语句:

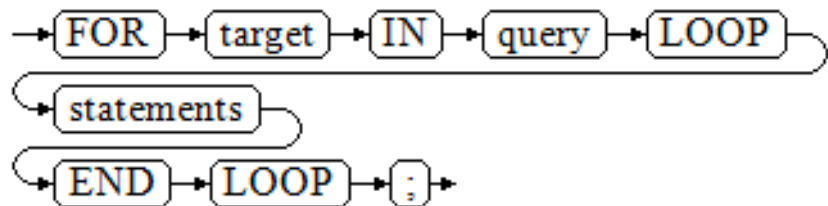


```
CREATE OR REPLACE PROCEDURE proc_for_iloop() AS
BEGIN
 FOR i IN 0..5 LOOP
 RAISE NOTICE '% time', i;
 END LOOP;
END;
/
CALL proc_for_iloop();
```



## 控制语句 - FOR\_LOOP查询语句 (2)

- FOR\_LOOP查询语句：

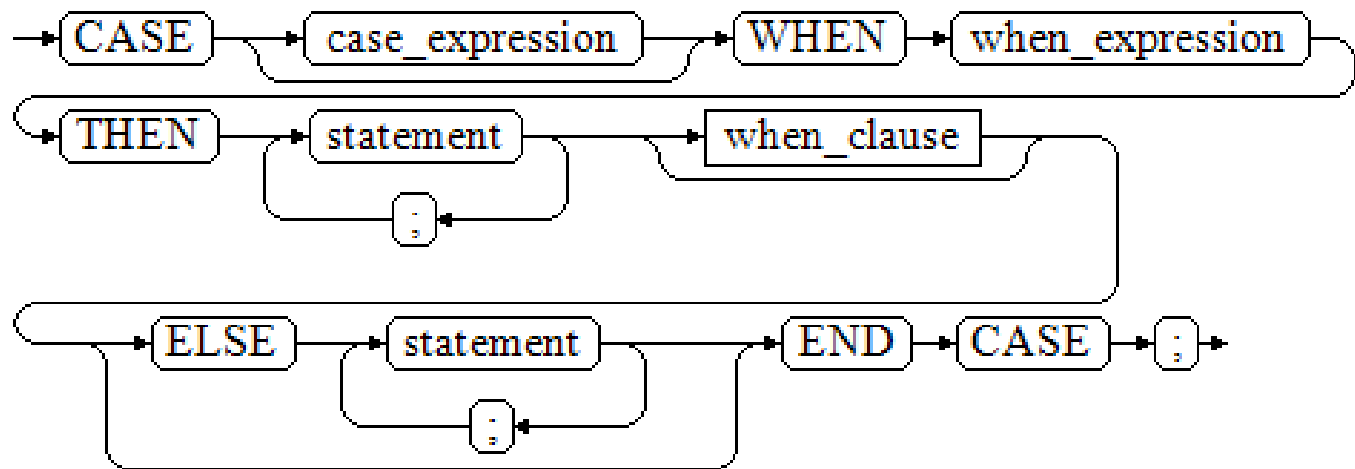


```
CREATE OR REPLACE PROCEDURE proc_for_loop()
AS
BEGIN
 FOR emp IN SELECT * FROM t_emp1
 LOOP
 RAISE NOTICE '%', emp.name; -- 打印员工姓名
 END LOOP;
END;
/
CALL proc_for_loop();
```

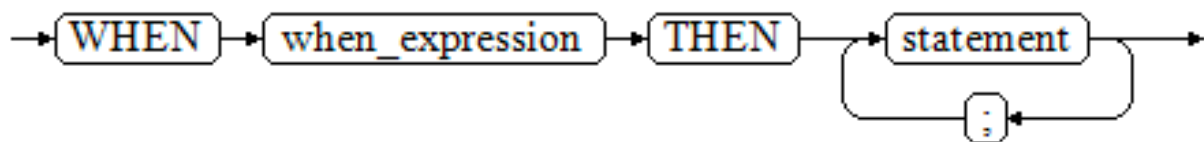


## 分支语句 (1)

- 分支语句 case when:



- 其中when\_clause子句为:
- case\_expression: 变量或表达式。
- when\_expression: 常量或者条件表达式。
- statement: 执行语句。





## 分支语句 (2)

```
CREATE OR REPLACE PROCEDURE proc_case_branch(pi_result in integer, pi_return out integer)
AS
BEGIN
 CASE pi_result
 WHEN 1 THEN
 pi_return := 111;
 WHEN 2 THEN
 pi_return := 222;
 WHEN 3 THEN
 pi_return := 333;
 WHEN 6 THEN
 pi_return := 444;
 END CASE;
 raise info 'pi_return : %',pi_return ;
 END;
/
CALL proc_case_branch(3,0);
```



# 思考题



## 本章总结

- 本章进一步讲述了SQL相关的高级语法的使用及应用程序接口开发、存储过程和触发器。高级语法主要从约束、JOIN、批处理、TRUNCATE、子查询及VACUUM操作等方面进行讲述；存储过程和触发器主要讲述了其概念、语法及应用等；应用程序开发主要介绍了常用的基于JDBC开发、ODBC开发、Psycopy开发等。



## 学习推荐

- openGauss开源社区
  - <https://opengauss.org/zh/>
- openGauss帮助文档（5.0.0版本）
  - <https://docs.opengauss.org/zh/docs/5.0.0/docs/GettingStarted/GettingStarted.html>



The background of the slide features a blue-tinted image of several business professionals in a modern office. They are standing on a highly reflective floor, and their silhouettes are clearly visible. The scene is set against a backdrop of large windows that look out onto a city skyline. The overall aesthetic is professional and corporate.

谢谢

[www.huawei.com](http://www.huawei.com)